

Generative AI: retrieval and evaluation

AI ENGINEERING: GENAI AND AGENTIC SYSTEMS

Today you build `nw/policy`, Project 3: heading-aware chunks that carry their metadata, hybrid retrieval with fusion and a reranker, answers whose citations are checked in code, three refusal paths, a harness that scores retrieval and generation apart, and the loop around it: versioned prompts, an index that knows what it was built from, a cache, feedback that becomes golden cases, drift, and a gate that runs free on every pull request.

Where we are

PART 1

Foundations and
the AI service layer

PART 2

Machine learning in
production

PART 3

Deep learning and
representation

PART 4

Generative AI:
retrieval and
evaluation

PART 5

Agentic AI systems

PART 6

Deployment,
observability and
capstone

Projects 1 and 2 classify. Nothing yet answers a question. Project 3 does, from twenty-eight policy documents, and the agent in the next part calls it as `search_policies`.

Today, step by step

- 1 Read the corpus and the contract
- 2 Tokenise sensitive fields before anything is indexed
- 3 Split on headings, carry the metadata, write the manifest
- 4 Hybrid retrieval with fusion, filters, and a reranker
- 5 Answer with citations enforced in code, or refuse, with a prompt version
- 6 The harness, retrieval and generation apart
- 7 The regression gate, and the free one in CI
- 8 Change one variable
- 9 The service, with a cache and a feedback endpoint
- 10 The service knows when the questions change
- 11 Session check and commit
- 12 Argue with the judge (optional)

28 documents, three superseded by a newer version and four internal. 77 golden cases, 17 of them must refuse.

Structured output is the integration contract

Mechanism	Fails when	Portable
Vendor JSON mode	long outputs, schemas with nested optionals	no
Tool calling as a schema	the model decides not to call the tool	no
Schema in the prompt, validation in code, one repair	rarely, and visibly	every vendor

Every answer and every judgement today is a validated object from `client.structured`. Nothing downstream ever reads prose.

Chunking is an engineering decision, and the index remembers it

Strategy	Keeps	Loses	Use when
Fixed windows	nothing to configure	headings, section boundaries, metadata	unstructured prose
Heading - aware	section, title, metadata per chunk	nothing important on structured docs	policies, runbooks, contracts
Sentence windows with overlap	local context	document structure	long narrative text

A chunk here is one heading's text, split only when too long, carrying document, section, effective date, audience and `superseded_by`. The build writes `manifest.json`: the corpus hash, the chunker parameters, both model names, 212 chunks, the prompt versions and a drift baseline. `build_index -check` exits 1 when the corpus has moved on.

Sensitive values never reach the index, the prompt or the log

```
"Contact ops@bluefreight.example  
about account NW-11498"
```

```
"Contact [EMAIL_1]  
about account [ACCOUNT_1]"
```

Same value, same token within one text. The model can still say "the customer at [ACCOUNT_1] asked twice".

Redaction runs inside `chunk_document`

Once step 2 passes, no chunk anywhere can carry an email, account id, card, phone or key, and a test proves it end to end.

The phone pattern requires a country code so that `2025-03-01` and `99.95` survive. Dates and percentages are the answers.

Fuse dense and BM25 first, filter second, rerank last

- 1 Dense candidates, 30 of them: finds "how fast do you respond" when the document says "acknowledgement time"; blurs `99.95` and `sla-2023`
- 2 Lexical candidates: the exact term every time, no paraphrase ever. Fused with `rrf`: each list adds `1 / (60 + rank)`, no weight to tune
- 3 Filter: drop superseded when `current_only`, drop internal when the audience is `customer`
- 4 Cross-encoder reranks the top `max(3k, 10)` of the pool
- 5 Return the top `k`

Why the filter comes after retrieval

A superseded document may be the best lexical match. Filter before fusion and it still wins and gets fused in. Metadata decides, not luck.

The reranker reads query and chunk together. Most of today's quality gain, and most of its latency.

Citations are enforced in code, not requested in the prompt

- 1 Tell the model which chunk ids it may cite
- 2 Check every citation against the ids that were actually in the context
- 3 Drop invented ids and count them
- 4 No valid citation left: this is an answer from memory. Refuse it

Deterministic, so exactly 1.0

Citation validity in the harness must be 1.000. If it is not, the code is wrong, not the model.

Refusal is an outcome, not an error

Path	Reason on the response	Model called
Retrieval too weak to bother the model	<code>weak_retrieval</code>	no
The model says the context does not cover it	<code>model_unanswerable</code>	yes
No valid citation survived the check	<code>no_valid_citation</code>	yes

The duty manager rule for P0 tickets exists only in the internal escalation matrix. A customer asking gets a refusal; the same question with `audience: internal` gets an answer with a citation into `escalation-matrix`.

A prompt is code the model runs, so it has a version

```
policy.answer@6c2999977051
```

The name and the first twelve hex characters of the SHA-256 of the text. One changed word, a new hash.

Registered once, next to the text: `ANSWER_PROMPT = register("policy.answer", SYSTEM)`.

- Travels on every Answer, refusals included, every harness row, `eval.json`, the index manifest and the model-call span as `nw.prompt_version`
- `python -m nw.llm.prompts` lists the four the course runs: the answer, the judge rubric, the structured wrapper, the agent's rules
- Two images built a week apart can run different prompts on one package version. The hash is what tells you

Evaluate retrieval and generation separately

Metric	Measures	Computed by
Recall@k	did the gold chunks make the list	deterministic
MRR	how high was the first gold chunk	deterministic
Faithfulness	is every claim supported by the cited passages	the Judge, against a rubric
Citation validity	were the citations real	deterministic, must be 1.0
Refusal correctness	refused when it should, answered when it should	deterministic
Latency, cost	per case	the shared cost meter

A bad answer from good retrieval is a prompt or model problem. A bad answer from bad retrieval is a chunking or index problem. One combined score hides which one you have.

A judge is an instrument, so calibrate it before you read it

0.8

Cohen's kappa on 30 human-labelled answers, false pass rate 0

- A different, stronger model than the workhorse, a rubric with a hash, a fixed version
- Agreement about 0.90 at the 0.75 threshold, false fail about 0.19, about 0.17 USD
- Agreement alone flatters a judge that passes everything; kappa corrects for chance
- The false pass rate bounds what the gate can detect. Recalibrate when the judge model, the rubric or the threshold changes; the report warns when the rubric's hash does

Pay for the judge once, gate on every run, gate for free on every pull request

Run	Calls	Gates on	Cost
judged	Workhorse, Judge	five metrics	about 6 USD, the baseline
Workhorse only	Workhorse	four	about 1 USD
retrieval only	none	recall@k, MRR	nothing, 8 seconds

Tolerance 0.03. Citation validity below 1.0 fails always. A baseline measured on another corpus or prompt is flagged before the verdict.

`eval-gate.yml`, two jobs

retrieval on every pull request that touches the corpus, the package, the prompts or the golden set: build the index, `--check`, the free harness against the committed baseline. No credentials.

gate on demand, a track and a judge flag, OpenID Connect into your account.

One variable, one row on the board

Variable	Flag
Chunk size 200	<code>build_index --max-tokens 200</code> , then evaluate
k = 2, 4 or 12	<code>--k 2</code> , <code>--k 4</code> , <code>--k 12</code>
Hybrid off	<code>--no-hybrid</code>
Reranker off	<code>--no-rerank</code>
Minimum score 0.2	<code>--min-score 0.2</code>

Retrieval variables first with `--retrieval-only`, eight seconds for nothing. Then `--no-judge`, about a dollar and a couple of minutes, for the row: which metric moved, by how much, in which direction, and what it cost in p95 latency.

Five things get a version, and none of them is a weights file

Versioned	Where	Changes when
Prompt	<code>nw/llm/prompts</code> , <code>name@hash</code> on every answer, row and span	one word of a prompt or rubric
Model id	by role in <code>nw/config.py</code> ; on every answer and report	the track, an override, a deprecation
Index manifest	corpus hash, chunker, embedder, reranker, chunk count, prompt versions, drift baseline	the corpus, the chunker, either model
Golden set	77 cases, 17 must refuse	feedback becomes cases
Baseline	one judged run with its prompt versions and corpus hash	a judged run you adopt

`/version` shows the manifest, `index_stale` , and which prompts changed since the build.
`nw_policy_index_stale` is the gauge; `index_stale` the log line.

The service closes the loop: a cache that cannot lie, feedback that becomes cases, drift that says when to look

Exact-match cache

Key: normalised question, audience, k, index hash, prompt version, model id. TTL and size, off by default. A rebuild or a prompt change empties it without a flush.

Not semantic: the Pro SLA is not the Enterprise SLA. The prompt cache on the system prompt is the cost lever, already on.

POST /feedback

`helpful`, `wrong` or `unsafe` on an `answer_id`, appended with the question, citations, confidence and prompt version.

`feedback --to-golden` prints the wrong and unsafe ones in the golden set's shape. A person writes the answer and commits the case.

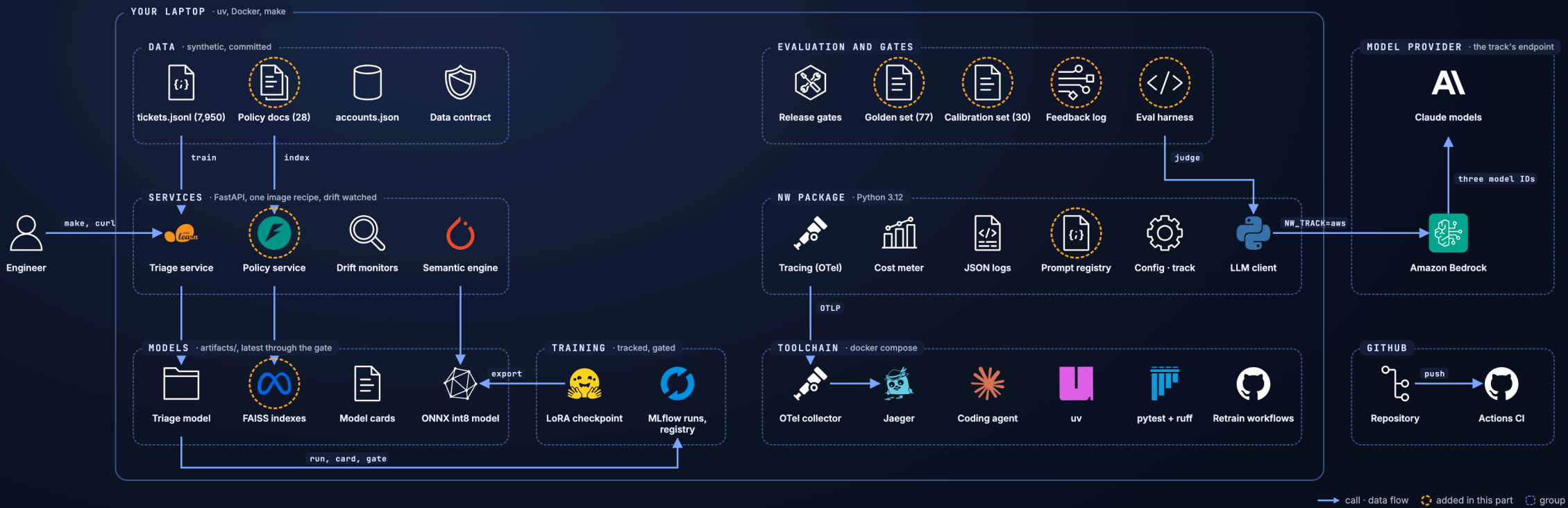
/drift

Top-hit confidence PSI against the golden questions, refusal rate against 0.221, answer length. Fifty "Login broken" questions: PSI 8.75, refusal rate 1.0, `drift_alert`, for nothing.

`build_index --capture` makes a captured week the baseline.

What exists after today, AWS track

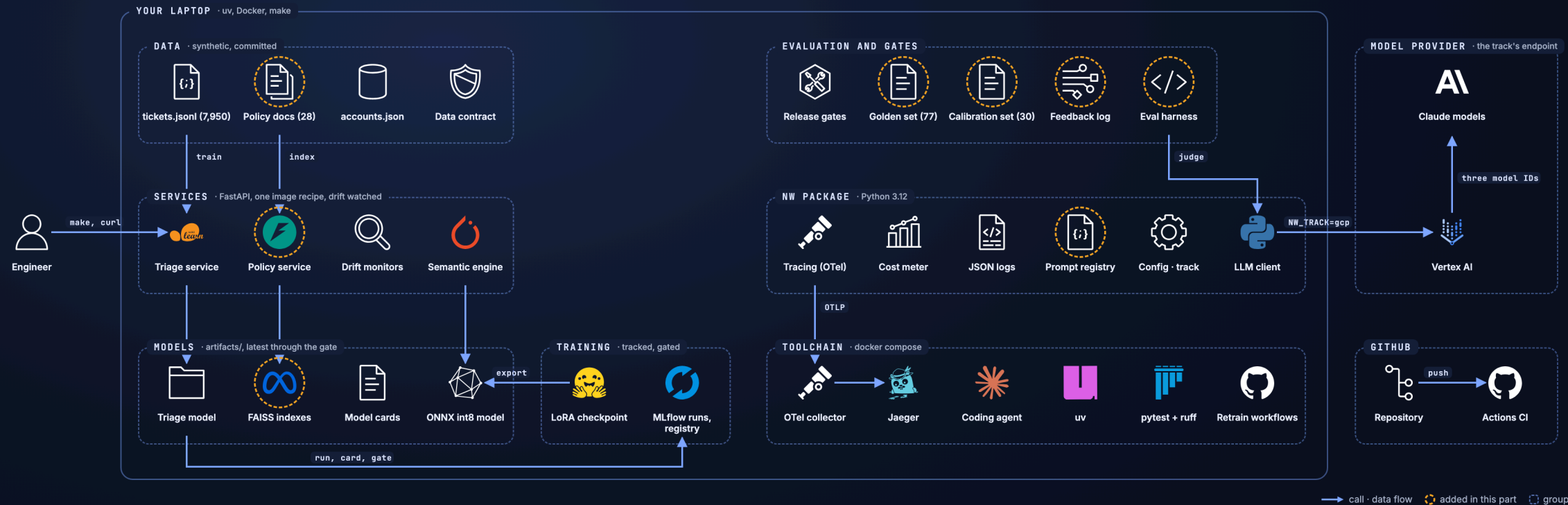
GENERATIVE AI: RETRIEVAL AND EVALUATION · AWS



The policy service joins triage and the semantic engine, with its FAISS index and manifest under artifacts. Ringed: the policy corpus, the golden set, the calibration set, the regression gate and the harness that judges through Bedrock.

What exists after today, Google Cloud track

GENERATIVE AI: RETRIEVAL AND EVALUATION · GOOGLE CLOUD



Same package, same harness, same gate; Vertex AI is the endpoint for the workhorse and the judge. The provider file is the only difference.

Acceptance criteria

1. Sensitive fields never reach a chunk; dates and percentages do
2. Superseded and internal chunks are filtered by metadata, after fusion
3. Citation validity exactly 1.0 on the golden set, and a prompt version on every answer
4. Recall@8 at or above 0.85 and correct refusal on at least 15 of the 17 must-refuse cases
5. A report with measured cost and p95 latency, a manifest that matches the corpus, and a gate that passes, fails when it should, and runs free on a pull request

47 passed

what `make session04` prints when you are done

Questions worth asking

- You cut k from 8 to 2. Which metric moves first, and in which direction?
- The judge scores 0.5 on an answer you think is correct. Who is right, and how would you find out?
- A customer asks a question whose answer is only in the internal escalation matrix. What should the service say, and what should the log say?
- Faithfulness rises on Monday after someone sharpened the prompt on Friday. Which three things in the report tell you it was the prompt?
- `/drift` alerts with the refusal rate at three times the baseline and a confidence PSI of 0.05. What moved?